

```

1  #include <iostream>
2  #include <vector>
3  #include <iomanip>
4  #include <cmath>
5
6  using namespace std;
7
8  void printMatrix(const vector<vector<double>>& A, const vector<double>& b, const
string& title) {
9      cout << "\n" << title << endl;
10     for (size_t i = 0; i < A.size(); i++) {
11         for (size_t j = 0; j < A[i].size(); j++) {
12             cout << setw(8) << fixed << setprecision(2) << A[i][j] << " ";
13         }
14         cout << " | " << setw(8) << b[i] << endl;
15     }
16     cout << endl;
17 }
18
19 vector<double> gaussianElimination(vector<vector<double>> A, vector<double> b) {
20     int n = A.size();
21
22     printMatrix(A, b, "Initial System:");
23
24     // Forward elimination
25     for (int i = 0; i < n; i++) {
26         cout << "--- Eliminating column " << i << " ---" << endl;
27
28         // Pivoting
29         int maxRow = i;
30         for (int k = i + 1; k < n; k++) {
31             if (fabs(A[k][i]) > fabs(A[maxRow][i])) {
32                 maxRow = k;
33             }
34         }
35
36         if (maxRow != i) {
37             swap(A[i], A[maxRow]);
38             swap(b[i], b[maxRow]);
39             printMatrix(A, b, "Swap row " + to_string(i) + " with row " +
to_string(maxRow));
40         }
41
42         // Normalize pivot row
43         double pivot = A[i][i];
44         for (int j = 0; j < n; j++) {
45             A[i][j] /= pivot;
46         }
47         b[i] /= pivot;
48
49         printMatrix(A, b, "Normalize row " + to_string(i));
50
51         // Eliminate below
52         for (int k = i + 1; k < n; k++) {
53             double factor = A[k][i];
54             for (int j = 0; j < n; j++) {
55                 A[k][j] -= factor * A[i][j];
56             }
57             b[k] -= factor * b[i];
58
59             printMatrix(A, b, "R" + to_string(k) + " = R" + to_string(k) + " - (" +
to_string(factor) + ")R" + to_string(i));
60         }
61     }
62
63     // Back substitution

```

```

64     vector<double> x(n);
65     cout << "---- Back Substitution ----" << endl;
66
67     for (int i = n - 1; i >= 0; i--) {
68         x[i] = b[i];
69         for (int j = i + 1; j < n; j++) {
70             x[i] -= A[i][j] * x[j];
71         }
72         cout << "x" << i << " = " << fixed << setprecision(4) << x[i] << endl;
73     }
74
75     return x;
76 }
77
78 int main() {
79     // Test system (3 equations)
80     vector<vector<double>> A = {
81         {1, -2, 6},
82         {2, 2, 3},
83         {-1, 0, 3}
84     };
85
86     vector<double> b = {0, 3, 2};
87     /*
88     // * Test system (4 equations)
89     vector<vector<double>> A = {
90         {2, -1, 1, 2},
91         {1, 1, -1, 1},
92         {3, -1, 2, 3},
93         {1, 2, 3, -1}
94     };
95
96     vector<double> b = {8, 2, 13, 4};
97     */
98     vector<double> solution = gaussianElimination(A, b);
99
100     cout << "\nFinal Solution:" << endl;
101     for (int i = 0; i < solution.size(); i++) {
102         cout << "x" << i << " = " << fixed << setprecision(4) << solution[i] <<
endl;
103     }
104
105     return 0;
106 }

```